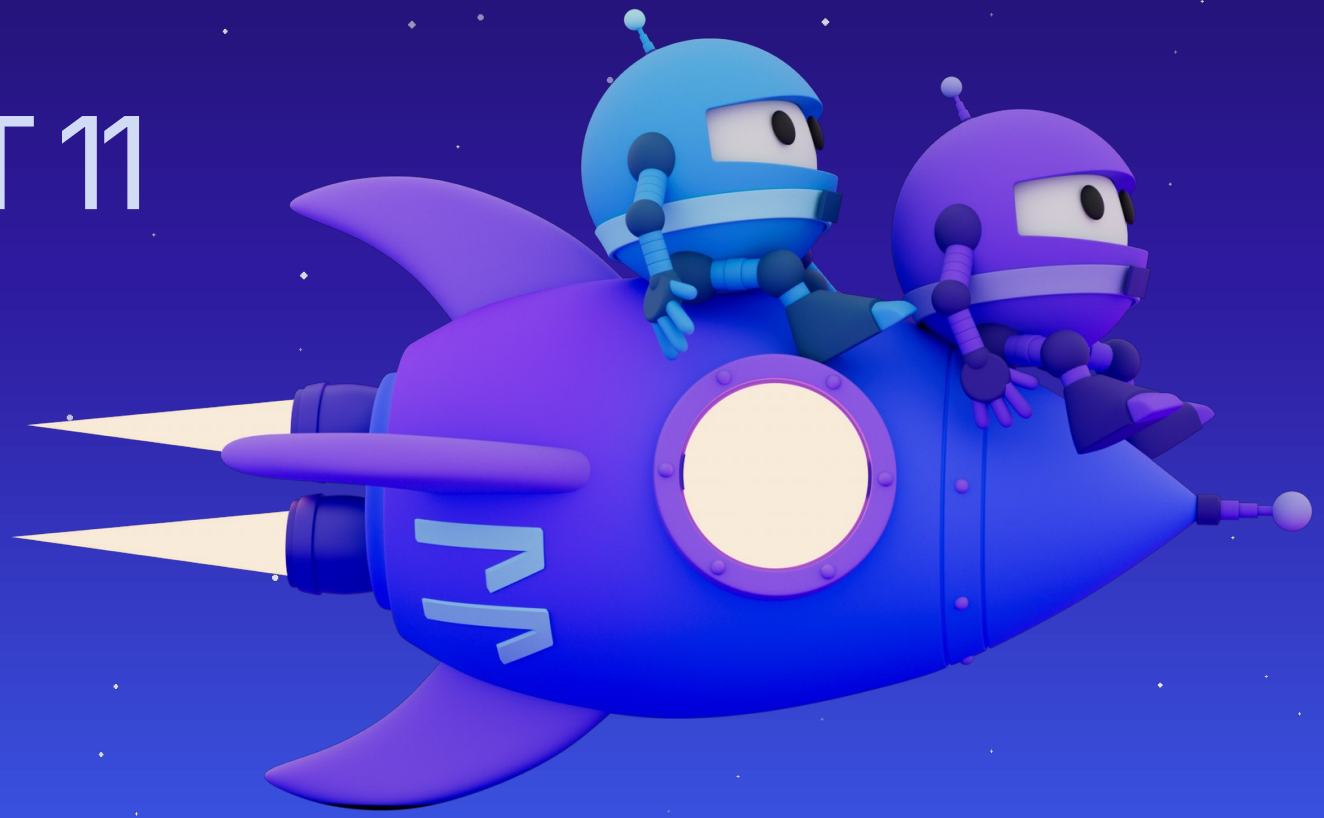
Infrastructure as (Actual) Code

with C# 15 and .NET 11

.NET



Cam Soper

Systems engineering, Pulumi

Infrastructure as code, they said

```
resource "azurerm_storage_account" "site" {
  name                     = "site${var.env}"
  resource_group_name      = "rg-${var.env}"
  location                 = "centralus"
  account_tier             = "Standard"
  account_replication_type = var.env == "prod" ? "ZRS" : "LRS"
}

resource "azurerm_management_lock" "keep" {
  count      = var.env == "prod" ? 1 : 0
  name       = "do-not-delete"
  scope      = azurerm_storage_account.site.id
  lock_level = "CanNotDelete"
}
```

That's not code.
That's config with
extra steps.



Cam Soper

Systems engineering, Pulumi

13 years at Microsoft: .NET and Azure, co-host of On .NET Live

At Pulumi: built and run our AI docs system, now building our agentic SDLC

What “actual code” gets you

Things C# devs already have

A compiler

Mistakes show up as red squiggles, not outages

Types and IntelliSense

F12 into your infrastructure. Autocomplete every property.

Tests

Unit-test the whole stack against a fake cloud

Packages

Share infrastructure on NuGet like any other library

Pulumi, in one slide

Your program declares the resources you want.

Pulumi diffs that against your last deployment, and makes it so.

Open source. No account needed:
`pulumi login --local`

```
var group = new ResourceGroup("demo");

var storage = new StorageAccount("site",
    new()
    {
        ResourceGroupName = group.Name,
        Kind = Kind.StorageV2,
        Sku = new SkuArgs
        {
            Name = SkuName.Standard_LRS,
        },
    });
```

01 Demo: Hello, cloud



C# 15: union types

A union is exactly one of a fixed set of types.

Miss a case in a switch, and the compiler tells you.

One .csproj line makes that a build error.

```
public record Dev;
public record Staging;
public record Prod(string OnCall);

public union Stage(Dev, Staging, Prod);

SkuName Redundancy(Stage s) => s switch
{
    Dev     => SkuName.Standard_LRS,
    Staging => SkuName.Standard_LRS,
    Prod    => SkuName.Standard_ZRS,
};
```

02 Demo: The compiler is your policy engine



I wrote a rule for my agents. In bold.

From my own CLAUDE.md:

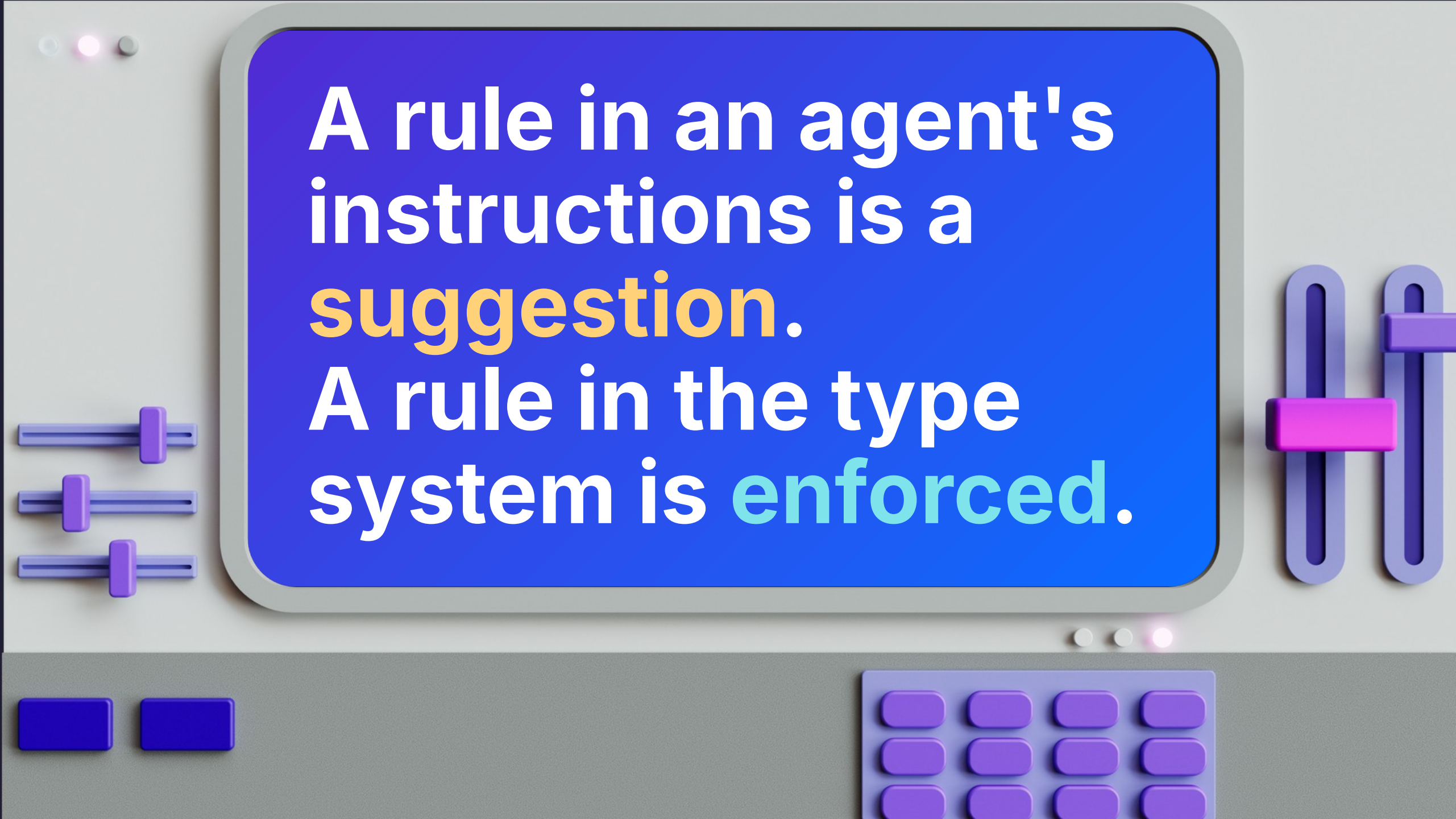
“Do NOT put Access in front of hades: it self-implements OAuth and would double-prompt.”

One auth posture per service. “Access and its own OAuth” can't be written.

```
public union McpAuth(
    SelfOAuth,
    AccessEdgeOnly,
    AccessWithOriginCheck);

record McpService(
    string Name, McpAuth Auth);

var hades = new McpService(
    "hades", new SelfOAuth());
// One service. One posture.
```


A rule in an agent's instructions is a suggestion.
A rule in the type system is enforced.

03 Demo: Agent vs. compiler



Take it home
Slides, code, and notes

`demos/`

All three demos, each one runnable on its own

`tests/`

The whole stack, tested against a fake cloud. No Azure needed.

`bonus/single-file/deploy.cs`

Everything in one file: dotnet run deploy.cs

Gotcha

File-based apps need #:property PublishAot=false

`github.com/CamSoper/infrastructure-as-actual-code`



Thank you

github.com/CamSoper/infrastructure-as-actual-code

pulumi.com/docs/iac/languages-sdks/dotnet

“What's new in C# 15” on Microsoft Learn

